MECH INTERP DOJO

Understanding the expressivity of multi-headed attention

One head computes AND and OR. Not XOR. Two heads can. Where does it end?

Intro meeting · June 2026

Mechanistic interpretability

“Reverse engineering neural networks, from the learned weights down to human-interpretable algorithms.”

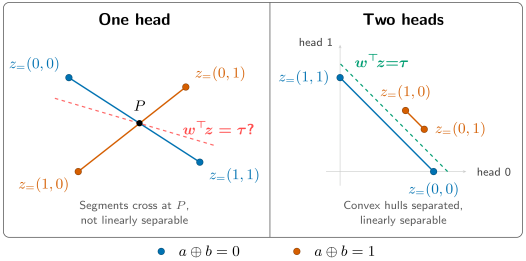
– Neel Nanda, mechanistic interpretability lead, Google DeepMind

- Knowing what a model computes inside is the basis for trusting and steering it.
- Today this is mostly empirical. We rarely get exact, provable statements about what a component can represent.

Our angle: start from the **smallest questions we can answer exactly**.

A crisp instance: XOR in one attention layer

How many heads does a single attention layer need for XOR? **Exactly two**: one head provably cannot, two can.

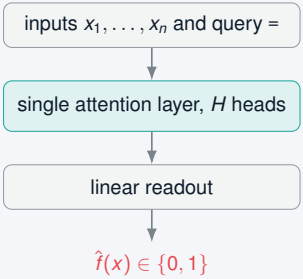


One attention head cannot compute XOR, but two can.

Figure: "How many attention heads do you need to do XOR?", LessWrong.

Scaling it up: from XOR to $H^*(f)$

$H^*(f)$: the **fewest attention heads** that compute a Boolean function $f : \{0,1\}^n \to \{0,1\}$, in a single attention layer with a linear readout.



- Proof of concept: $H^*(\mathrm{parity}_n) = n$, while $H^*(\mathrm{AND}) = H^*(\mathrm{OR}) = 1$.
- A complexity measure native to transformers.
- The blog's obstruction is the $n = 2$ case of our **checkerboard lemma**.

What we know so far

Lower bounds

$H^*(f) \geq 2$ if f has a checkerboard

$\deg_{\pm}(f) \leq H^*(f)$

Upper bounds

$H^*(\mathrm{AND}) =$
$H^*(\mathrm{OR}) =$
$H^*(\mathrm{MAJ}) = 1$

$H^*(f) \leq M - 1$
(weighted sums)

Exact

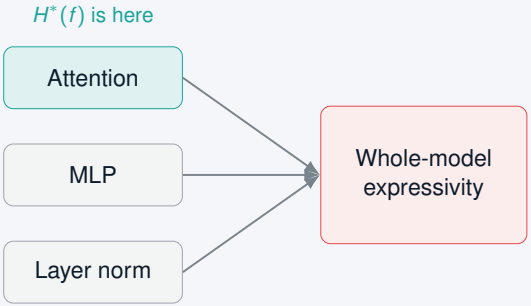
$H^*(\mathrm{XOR}_n) = n$

parity is extremal: one head per bit

We can bound many families, but no known invariant pins down $H^*(f)$ exactly. Finding one is the central open problem.

The grand goal

- Attention heads are one component. The grand goal: characterize **each** transformer component, then chain them together.
- A compositional theory of what transformers can and cannot represent.
- $H^*(f)$ is the **first brick**.



Why now: LLMs change the economics of math

LLMs can do real math

GPT-5 produced solutions to open Erdős problems, several later checked in Lean (2025).

LLMs can write Lean

Galois, *"Claude Can (Sometimes) Prove It"*: real proofs by **directing a coding agent**, barely any hand written Lean.

Proving results at this scale is finally within reach.

The fix: a verifiable Mech Interp Dojo

- LLM proofs hallucinate. **Lean 4 + mathlib** checks every one; the kernel rejects what is wrong.
- LLM drafts, Lean verifies. Even those Erdős proofs were Lean checked first.
- Deliverable: the **Mech Interp Dojo**, a Lean library and prompting scaffold. Agents do the proof engineering; humans steer.

```
theorem xor_needs_two :
  Hstar (xor 2) = 2 := by
  -- formal proof
```

✓ checked by Lean

Where you come in

- **Theory and proofs** push the bounds. `lemmas/`
- **Lean and the dojo** keep proofs composable. `head-complexity/`
- **Experiments** estimate $H^*(f)$ numerically. `src/hstar/`

Helpful: some Lean and coding agents. Mech interp optional.

Pick a track. Come build it with us.